

```

import asyncio
import logging
import os

from aiogram import Bot, Dispatcher, F
from aiogram.filters import CommandStart, Command
from aiogram.fsm.context import FSMContext
from aiogram.fsm.state import State, StatesGroup
from aiogram.fsm.storage.memory import MemoryStorage
from aiogram.types import Message
from dotenv import load_dotenv

import database as db

load_dotenv()

BOT_TOKEN = os.getenv("BOT_TOKEN")
ADMIN_IDS = [int(x) for x in os.getenv("ADMIN_IDS", "").split(",") if x.strip()]
BACKUP_CHAT_ID = os.getenv("BACKUP_CHAT_ID") or None # id приватного канала-хран

logging.basicConfig(level=logging.INFO)

bot = Bot(token=BOT_TOKEN)
dp = Dispatcher(storage=MemoryStorage())

class Registration(StatesGroup):
    waiting_for_name = State()

WELCOME_TEXT = (
    "Добро пожаловать! 🤖\n\n"
    "Здесь вы можете поделиться фото и видео со свадьбы.\n\n"
    "Для начала, пожалуйста, напишите ваше имя "
    "(можно просто ваш юзернейм) ."
)

QUALITY_REMINDER = (
    "Спасибо, {name}! 🙏\n\n"
    "⚠️ Важно: чтобы фото и видео не потеряли качество при отправке, "
    "загружайте их как ФАЙЛ (документ), а не как обычное фото/видео.\n\n"
    "Как это сделать в Telegram:\n"
    "1. Нажмите на скрепку 📎\n"
    "2. Выберите «Файл» (Document)\n"

```

```

"3. Найдите фото/видео в галерее и отправьте в исходном качестве\n\n"
"Если отправить как обычное фото или видео — Telegram сожмёт файл, "
"и качество будет хуже.\n\n"
"Теперь можно присылать ваши фото и видео! 📸🎥"
)

@dp.message(CommandStart())
async def cmd_start(message: Message, state: FSMContext):
    guest = await db.get_guest(message.from_user.id)
    if guest:
        await state.clear()
        await message.answer(
            f"С возвращением, {guest['name']}! Можете продолжать отправлять "
            f"фото и видео 📸 (не забывайте отправлять их как файл)."
        )
    return
    await state.set_state(Registration.waiting_for_name)
    await message.answer(WELCOME_TEXT)

@dp.message(Registration.waiting_for_name, F.text)
async def process_name(message: Message, state: FSMContext):
    name = message.text.strip()
    if not name:
        await message.answer("Пожалуйста, введите имя текстом.")
        return
    guest = await db.create_guest(
        telegram_user_id=message.from_user.id,
        name=name,
        username=message.from_user.username,
    )
    await state.clear()
    await message.answer(QUALITY_REMINDER.format(name=guest["name"]))

async def _handle_media(message: Message, file_id: str, file_unique_id: str, file
    guest = await db.get_guest(message.from_user.id)
    if not guest:
        await message.answer(
            "Пожалуйста, сначала отправьте /start и укажите своё имя, "
            "прежде чем присылать фото или видео."
        )
    return

backup_chat_id = None
backup_message_id = None

```

```

if BACKUP_CHAT_ID:
    try:
        caption = f"От: {guest['name']} (@{message.from_user.username or 'без'
        if file_type == "photo":
            sent = await bot.send_photo(BACKUP_CHAT_ID, file_id, caption=capt
        elif file_type == "video":
            sent = await bot.send_video(BACKUP_CHAT_ID, file_id, caption=capt
        else:
            sent = await bot.send_document(BACKUP_CHAT_ID, file_id, caption=c
        backup_chat_id = str(BACKUP_CHAT_ID)
        backup_message_id = sent.message_id
    except Exception as e:
        logging.error(f"Не удалось переслать в канал-хранилище: {e}")

await db.save_media(
    guest_id=guest["id"],
    telegram_user_id=message.from_user.id,
    file_id=file_id,
    file_unique_id=file_unique_id,
    file_type=file_type,
    backup_chat_id=backup_chat_id,
    backup_message_id=backup_message_id,
)
count = await db.count_media_for_guest(guest["id"])
await message.answer(f"Принято! ✅ Всего файлов от вас: {count}")

@dp.message(F.document)
async def handle_document(message: Message):
    doc = message.document
    mime = doc.mime_type or ""
    if mime.startswith("image/"):
        file_type = "photo"
    elif mime.startswith("video/"):
        file_type = "video"
    else:
        await message.answer(
            "Этот файл не похож на фото или видео. Пожалуйста, присылайте только
        )
        return
    await _handle_media(message, doc.file_id, doc.file_unique_id, file_type)

@dp.message(F.photo)
async def handle_photo(message: Message):
    largest = message.photo[-1]

```

```

    await _handle_media(message, largest.file_id, largest.file_unique_id, "photo")
    await message.answer(
        "💡 В следующий раз отправляйте фото как файл (📎 → Файл), "
        "чтобы сохранить исходное качество."
    )

@dp.message(F.video)
async def handle_video(message: Message):
    video = message.video
    await _handle_media(message, video.file_id, video.file_unique_id, "video")
    await message.answer(
        "💡 В следующий раз отправляйте видео как файл (📎 → Файл), "
        "чтобы сохранить исходное качество."
    )

@dp.message(Command("stats"))
async def cmd_stats(message: Message):
    if message.from_user.id not in ADMIN_IDS:
        return
    guests_count, media_count = await db.stats()
    await message.answer(f"Гостей зарегистрировано: {guests_count}\nВсего файлов:

@dp.message()
async def fallback(message: Message, state: FSMContext):
    current_state = await state.get_state()
    if current_state == Registration.waiting_for_name.state:
        return
    guest = await db.get_guest(message.from_user.id)
    if not guest:
        await state.set_state(Registration.waiting_for_name)
        await message.answer(WELCOME_TEXT)
    else:
        await message.answer("Отправьте, пожалуйста, фото или видео 📸🎥 (лучше в

```

```

async def main():
    await db.init_db()
    await bot.delete_webhook(drop_pending_updates=True)
    await dp.start_polling(bot)

if __name__ == "__main__":
    asyncio.run(main())

```

